Concurrent and Distributed Systems

Lab 4 - Deadlock

Source code

Download the archive **cds.lab4.zip**

Exercise 1

In this task, you will recreate the deadlock scenarios discussed in the lecture.
Note: Deadlocks may not appear on every run — this is expected due to non-deterministic thread scheduling.

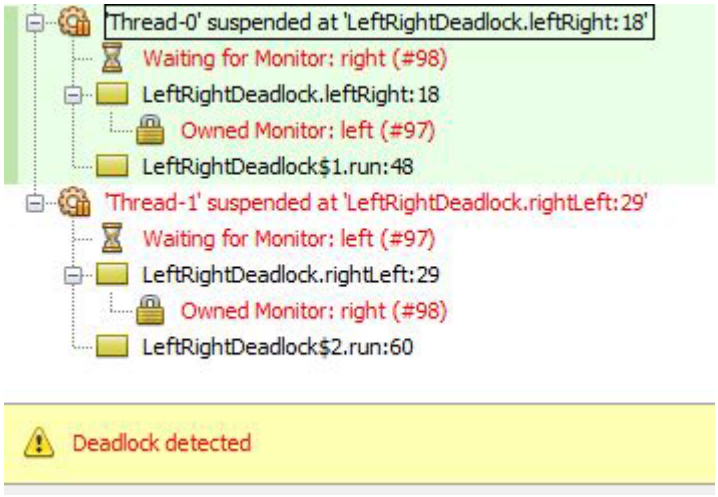
Instructions

- Run the example programs provided in the **cds.lab4.zip** archive.
- Focus on the implementations known to be deadlock-prone:
  - `LeftRightDeadlock.java`
  - `DeadlockPhilosopher.java`

  Modify the code to make deadlock more likely, for example uncomment `Thread.sleep(50)`, and add log messages to observe thread behaviour.

- To assist with debugging, refer to **Java_9_Concurrency_Cookbook_Chapter_9_Testing_Concurrent_Applications.pdf**, section **Configuring NetBeans for debugging concurrency code** (page 467).
  You can learn how to:
  - Set breakpoints
  - Suspend/resume individual threads
  - Use the **Check for Deadlock** option in the Debug menu
  - Change the default Options in the Tools menu for the Java Debugger

  For example, when debugging `LeftRightDeadlock.java` using **Check for Deadlock** should reveal when two threads are stuck waiting on each other (deadlock).



Exercise 2

Implement a version of the Dining Philosophers problem using the **global order** strategy discussed in the lecture (see slide 33). You may adapt an existing solution such as `FixedPhilosopher2.java`. This strategy avoids deadlock by ensuring chopsticks are always picked up in a globally consistent order.

Exercise 3

Explore several alternative solutions to the Dining Philosophers problem from Butcher (2014) Seven Concurrency Models in Seven Weeks: When Threads Unravel. Slides 34–37 summarise key approaches. All the programs from the book are available here: https://github.com/islomar/seven-concurrency-models-in-seven-weeks/tree/master/ThreadsLocks. Compare the solutions – what are the advantages/disadvantages of each of them?

Exercise 4

The solutions in the lecture (`FixedPhilosopher1.java, FixedPhilosopher2.java`) avoid deadlock but may still allow starvation — a philosopher could go a long time without eating if neighbours happen to be faster or greedier.

Task

Extend an existing solution by adding:

- A variable tracking **hunger**
- A timestamp for the **last time each philosopher ate**

If a philosopher is hungry and has not eaten for longer than their neighbour, then the neighbour **should not** pick up their shared chopstick.

Your enhanced solution must:

- **Avoid deadlock**
- **Improve fairness** so no philosopher is indefinitely starved

Optional enhancement

Model philosophers with different eating durations, for example:

```
private void eat() throws InterruptedException {
```

```
  // Long duration = (long) (Math.random() * 1000);
  Long duration = (long) (Math.random() * 1000) * (seat+1);
  System.out.println("Philosopher " + seat + " eats for " + duration + " ms.");
  Thread.sleep(duration);
}
```

Resources

- J. Fernandez Gonzalez (2017), **Java 9 Concurrency Cookbook** (2nd edition), Packt Publishing. You can access the online version through the library (log in with your university account), link here. The GitHub repository providing the complete Java classes for each book chapter example is here: https://github.com/PacktPublishing/Java-9-Concurrency-Cookbook-Second-Edition
- https://github.com/islomar/seven-concurrency-models-in-seven-weeks/tree/master/ThreadsLocks